Prompt Engineering

A Practical Guide to Designing Better Instructions for AI Models

What this guide covers

- What prompts are and what prompt engineering means
- The anatomy of a reliable prompt
- Context, constraints, examples, and output formats
- Few-shot prompting, roles, and task decomposition
- Prompting for RAG and tool-using agents
- Common prompt failures and debugging
- Evaluation and iterative improvement

Core idea: Prompt engineering is the systematic design, testing, and improvement of instructions and context so an AI model produces useful and consistent results.

1. What Is Prompt Engineering?

A **prompt** is the input given to an AI model. It can contain instructions, questions, context, examples, constraints, data, or a desired output format.

Prompt engineering is the practice of designing and refining these inputs to make model behavior more useful and predictable.

A weak prompt leaves important decisions to the model. A strong prompt reduces ambiguity by explicitly defining the task, relevant context, constraints, and expected output.

Example

Weak: "Explain RAG."

Stronger: "Explain Retrieval-Augmented Generation to a beginner software engineer. First define retrieval, embeddings, and vector databases in simple language. Then show the RAG flow and give one practical example."

Why the second prompt works better

- It identifies the audience.
- It defines the explanation level.
- It specifies concepts to cover.
- It specifies an order.
- It asks for a practical example.

Prompt engineering is therefore closer to **specification writing** than to writing clever questions.

Important limitation: Better prompts cannot guarantee correct answers. Model quality, context, retrieval, tools, and evaluation also matter.

2. Anatomy of a Strong Prompt

A useful prompt can contain several components. Not every task needs every component, but this structure is a strong starting point.

Component	Purpose	Example
Role / perspective	Sets useful behavior	You are a Python tutor.
Task	States what to do	Explain list comprehensions.
Context	Provides needed information	Learner knows loops but not comprehensions.
Constraints	Defines boundaries	Use simple language.
Output format	Controls answer shape	Return 3 sections and one example.
Examples	Shows desired pattern	Input → classification → explanation.

Reusable template

```
Role: You are a [role].
Task: [specific task].
Context: [relevant information].
Constraints: [rules, limits, exclusions].
Output: [required structure or format].
Quality: [what a good answer must contain].
```

The goal is not to make every prompt long. The goal is to make the **important requirements explicit**.

3. Core Prompting Techniques

Different tasks benefit from different techniques. These are tools, not rules that must always be used.

Zero-shot prompting

Ask the model to perform a task without examples. It is simple and often sufficient for straightforward tasks.

Few-shot prompting

Provide examples of desired input-output behavior. This is useful when the pattern is difficult to describe with instructions alone.

```
Positive → "I loved the movie"
Negative → "The movie was boring"
Neutral → "The movie was released in 2024"

Classify: "The acting was excellent."
```

Role prompting

Give the model a useful perspective such as "You are a code reviewer." A role can shape terminology and priorities, but it does not magically grant expertise or private knowledge.

Output constraints

```
Return JSON with exactly these keys:
"category", "confidence", "reason"
```

Decomposition

Break a complex task into smaller stages, such as extract facts → identify issues → produce a result. For production systems, concise checks or structured summaries are preferable to requesting hidden chain-of-thought.

4. Context, Grounding, and RAG

A model's answer depends heavily on the information available in its context. This makes prompt design especially important when an application supplies retrieved documents.

Typical RAG flow

- User submits a question.
- Retriever finds relevant document chunks.
- Chunks are placed into the model context.
- The prompt tells the model how to use that context.
- The model produces the final response.

Example RAG instruction

```
Answer the user's question using the provided context.
If the context does not contain enough information, say so.
Do not invent facts.

Context:
{context}

Question:
{question}
```

The prompt does not replace retrieval. If the retriever returns irrelevant or incomplete chunks, even a carefully written prompt may produce a poor answer.

Engineering lesson: RAG quality is a pipeline problem. Evaluate retrieval and generation separately.

Metadata filters can constrain retrieval to selected documents, tenants, pages, or other application-defined scopes.

5. Prompting Tool-Using Agents

An AI agent may use tools such as search, calculators, databases, APIs, or document retrieval. The prompt should define when tools are appropriate and how their results should be handled.

Example tool policy

```
Use search_doc when the answer may exist in uploaded documents.
Use calculator for arithmetic rather than estimating.
If a tool returns no relevant information, do not fabricate an answer.
Explain briefly when available sources are insufficient.
```

Good agent prompts clarify

- Which tool is appropriate for which task
- What each tool expects
- What each tool returns
- What to do when a tool fails
- When a tool should not be used
- How tool results affect the final response

Tool descriptions matter too. A vague description such as "Search documents" gives the model little guidance. A better description states what is searched, when to use it, and what it returns.

Example: "Search the uploaded PDFs for passages relevant to the user's question. Returns matching chunks with source and page metadata. Use this tool when the answer may be contained in the uploaded documents."

Application code should enforce security boundaries. A prompt should never be the only mechanism protecting private data or privileged tools.

6. Common Prompt Failures

Prompt debugging is often more useful than endlessly adding instructions. When output is wrong, first identify which part of the system failed.

Ambiguity — The task has multiple interpretations.

Fix: Define audience, scope, and desired result.

Missing context — Required information is absent.

Fix: Provide relevant context or retrieve it.

Conflicting instructions — Requirements contradict each other.

Fix: Remove contradictions and define priority.

Overly broad task — Too many decisions are left unstated.

Fix: Break the task into explicit stages.

Unstable format — Free-form output varies.

Fix: Specify a schema or exact fields.

Hallucination — The model fills gaps with unsupported content.

Fix: Ground the answer and allow "not enough information".

Bad retrieval — RAG supplies irrelevant chunks.

Fix: Improve chunking, embeddings, filters, or search strategy.

Practical debugging order

- Inspect the exact user query.
- Inspect the prompt actually sent to the model.
- Inspect retrieved context or tool results.
- Check output requirements.
- Change one variable at a time and test again.

7. Evaluation and Practical Workflow

A prompt is not finished because it sounds good. It is finished when it performs reliably on representative examples.

Build an evaluation set

Create realistic tests covering normal questions, ambiguous questions, missing information, edge cases, and adversarial inputs.

Measure what matters

- Correctness: Is the answer factually supported?
- Relevance: Does it answer the actual question?
- Grounding: Is it based on supplied sources?
- Format: Does it follow the required schema?
- Consistency: Is behavior reliable across similar inputs?
- Safety: Does it avoid exposing restricted or unsupported information?

Iterative workflow

```
1. Define the task and success criteria
2. Write the simplest adequate prompt
3. Create representative test cases
4. Run the prompt and inspect failures
5. Identify the actual cause
6. Change the prompt or pipeline
7. Re-run the evaluation set
8. Keep the change only if results improve
```

Final checklist

- Is the task explicit?
- Does the model have the necessary context?
- Are important constraints clear?
- Is the output format defined?
- Are unsupported answers handled explicitly?
- For RAG: is retrieval returning the right chunks?
- For agents: are tool-use rules and failure behavior clear?
- Have you tested the prompt rather than judging it from one example?

Bottom line: Prompt engineering is an engineering discipline: define behavior, provide the right context, constrain outputs, test systematically, and improve based on observed failures.